

Guest: Landing Page and Dynamic Content

Test Case Task ID: #1
Test Scenario: Verify that the landing page text is loaded from the database rather than fixed in the page code
Prerequisites: 1. Landing content has been saved by an admin 2. Database can be accessed
Test Steps: 1. Guest opens the site home page 2. Guest compares the headline and description text against the values held in the landing content table
Test Data: 1. The saved landing content records
Expected Results: The home page displays the text stored in the database, and no heading or description is hardcoded in the page.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-GST-001, TestLandingContentApiServlet.returnsContentMap

Test Case Task ID: #2
Test Scenario: Verify that the landing page still renders when the content lookup fails
Prerequisites: 1. The landing content source is unavailable or returns an error
Test Steps: 1. Guest opens the site home page while the content lookup is failing
Test Data: 1. A failing landing content request
Expected Results: The home page loads with its default text and no error page is shown to the guest.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-GST-002, TestLandingContentApiServlet.failsSoftOnDatabaseError

Test Case Task ID: #3
Test Scenario: Verify that an admin can change the landing page text without a code change
Prerequisites: 1. Admin is logged in 2. Database can be accessed
Test Steps: 1. Admin opens the landing content editor 2. Admin edits the headline and description fields 3. Admin clicks Save 4. Admin opens the public home page
Test Data: 1. A new headline value 2. A new description value
Expected Results: The new values are saved and the public home page shows the updated text straight away.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-GST-003, TestAdminLandingContentApiServlet.savesSubmittedFields

Test Case Task ID: #4
Test Scenario: Verify that landing page text containing HTML markup is rejected
Prerequisites: 1. Admin is logged in
Test Steps: 1. Admin opens the landing content editor 2. Admin enters a value that contains HTML markup 3. Admin clicks Save
Test Data: 1. A headline value containing an HTML script tag
Expected Results: The save is refused with a validation message, the value is not stored, and the public page is unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-GST-004, TestAdminLandingContentApiServlet.rejectsMarkup

Guest: Search, Filters and Browsing

Test Case Task ID: #5
Test Scenario: Verify that a guest can search listings by keyword without logging in
Prerequisites: 1. At least one active listing matches the keyword 2. No user is logged in
Test Steps: 1. Guest types a keyword into the search box 2. Guest presses Enter
Test Data: 1. The keyword laptop
Expected Results: The results page lists the active listings that match the keyword.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SRC-001, TestSearchServlet.testValidKeywordReturnsResults

Test Case Task ID: #6
Test Scenario: Verify that a search keyword cannot be used to inject SQL
Prerequisites: 1. Database can be accessed
Test Steps: 1. Guest types an SQL injection string into the search box 2. Guest presses Enter
Test Data: 1. The keyword ' OR '1'='1
Expected Results: The string is treated as ordinary search text. No extra listings are returned and no database error is shown.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SRC-002, TestSearchServlet.testSqlInjectionAttemptsIsHandledSafely

Test Case Task ID: #7
Test Scenario: Verify that a search keyword of exactly the maximum permitted length is accepted
Prerequisites: 1. Database can be accessed
Test Steps: 1. Guest types a keyword of exactly the maximum permitted length into the search box 2. Guest presses Enter
Test Data: 1. A keyword at the maximum permitted character length
Expected Results: The search runs normally and the results page is displayed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SRC-003, TestSearchServlet.testQueryAtMaxLength

Test Case Task ID: #8
Test Scenario: Verify that the sort option cannot be tampered with to inject SQL
Prerequisites: 1. A search results page is open
Test Steps: 1. Guest edits the sort value in the page address to an SQL fragment 2. Guest reloads the page
Test Data: 1. The sort value ; DROP TABLE auction
Expected Results: The unrecognised sort value is ignored, the results fall back to the default order, and no database error occurs.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SRC-004, TestSearchServletSort.testSqlInjectionReturnsDefault

Test Case Task ID: #9
Test Scenario: Verify that price, condition, location and ending time filters can be applied together
Prerequisites: 1. Listings exist across several prices, conditions and locations
Test Steps: 1. Guest opens the search results page 2. Guest sets a price range 3. Guest selects an item condition 4. Guest selects a location 5. Guest selects an ending time window 6. Guest applies the filters
Test Data: 1. A minimum and maximum price 2. An item condition 3. A location 4. An ending time window
Expected Results: Only listings satisfying all four filters are listed, and every applied filter is shown on the page.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SRC-005, TestSearchServletFilters.testAllFiltersCombined

Guest: Auction Detail and Seller Profile

Test Case Task ID: #10
Test Scenario: Verify that an unregistered visitor can view the bid history of an auction
Prerequisites: 1. An auction with at least one bid exists 2. No user is logged in
Test Steps: 1. Guest opens an auction detail page 2. Guest opens the bid history panel
Test Data: 1. An existing auction that has bids
Expected Results: The bid history is displayed and the guest is not redirected to the login page.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-DET-001, TestAuctionBidHistory.testPublicAccessNoSession

Test Case Task ID: #11
Test Scenario: Verify that bidder identities are masked in the public bid history
Prerequisites: 1. An auction has bids from several different users
Test Steps: 1. Guest opens the auction detail page 2. Guest opens the bid history panel
Test Data: 1. An auction with bids from at least three users
Expected Results: Every bidder name and email is shown in masked form, and no full name or full email address appears.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-DET-002, TestAuctionBidHistory.testSecurityUtilMaskingRules

Test Case Task ID: #12
Test Scenario: Verify that requesting an auction that does not exist is handled cleanly
Prerequisites: 1. Database can be accessed
Test Steps: 1. Guest edits the auction identifier in the page address to one that does not exist 2. Guest loads the page
Test Data: 1. An auction identifier that is not in the database
Expected Results: A not found page is shown. No error trace or internal detail is displayed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-DET-003, TestAuctionBidHistory.testUnknownAuction404

Test Case Task ID: #13
Test Scenario: Verify that a public seller profile does not expose the seller's real email address
Prerequisites: 1. A seller with completed sales exists
Test Steps: 1. Guest opens a listing by that seller 2. Guest clicks the seller name to open the public profile
Test Data: 1. A seller account with completed sales
Expected Results: The profile shows a masked email address only, and the full address does not appear anywhere on the page.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-DET-004, TestSellerProfileServlet.testMaskedEmailInProfile

Account: Registration and Login

Test Case Task ID: #14
Test Scenario: Verify that a visitor can create an account
Prerequisites: 1. The chosen username and email are not already registered 2. Database can be accessed
Test Steps: 1. Guest opens the registration page 2. Guest fills in every required field 3. Guest clicks Register
Test Data: 1. An unused username 2. An unused email address 3. A password meeting the complexity policy
Expected Results: The account is created and the visitor is taken to the login page.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REG-001, TestRegisterServlet.testInsert

Test Case Task ID: #15
Test Scenario: Verify that a registrant cannot give themselves the admin role during registration
Prerequisites: 1. The chosen username and email are not already registered
Test Steps: 1. Guest opens the registration page 2. Guest adds a role field set to ADMIN to the submitted form 3. Guest clicks Register 4. Admin checks the new account in the user list
Test Data: 1. A registration form carrying role set to ADMIN
Expected Results: The account is created as an ordinary member. The submitted role is ignored and no admin rights are granted.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REG-002, TestRegisterServlet.testSuppliedRoleIsIgnored

Test Case Task ID: #16
Test Scenario: Verify that a password which fails the complexity policy is refused at registration
Prerequisites: 1. The chosen username and email are not already registered
Test Steps: 1. Guest opens the registration page 2. Guest enters a weak password 3. Guest clicks Register
Test Data: 1. A password that does not meet the complexity policy
Expected Results: Registration is refused with a validation message and no account is created.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REG-003, TestRegisterServlet.testPasswordValidation

Test Case Task ID: #17
Test Scenario: Verify that a registered member can log in
Prerequisites: 1. An active account exists with a known password
Test Steps: 1. Member opens the login page 2. Member enters their email and password 3. Member clicks Login
Test Data: 1. A registered email address 2. The matching password
Expected Results: The member is logged in and taken to the member area, with their name shown in the navigation bar.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REG-004, TestLoginServlet.testSuccessfulLoginSetsSession

Test Case Task ID: #18
Test Scenario: Verify that a suspended account cannot log in even with the correct password
Prerequisites: 1. An account exists whose status is SUSPENDED
Test Steps: 1. Member opens the login page 2. Member enters the correct email and password of the suspended account 3. Member clicks Login
Test Data: 1. The email and correct password of a suspended account
Expected Results: Login is refused with a message explaining the account is suspended, and no session is created.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REG-005, TestLoginServlet.testSuspendedUserCannotLogin

Test Case Task ID: #19
Test Scenario: Verify that repeated failed logins lock the account for a cooldown period
Prerequisites: 1. An active account exists with a known password
Test Steps: 1. Member enters the correct email with a wrong password and clicks Login 2. Member repeats this until the configured attempt limit is reached 3. Member enters the correct password and clicks Login
Test Data: 1. A registered email address 2. Wrong passwords repeated up to the attempt limit 3. The correct password
Expected Results: After the limit is reached the account is locked and the remaining cooldown is shown. The correct password is also refused while the lock is in force.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REG-006, TestAuthApiServlet.loginLocksOutAfterThreshold

Account: Password Reset, OTP and Two-Factor Authentication

Test Case Task ID: #20
Test Scenario: Verify that a member can reset a forgotten password using a one-time code
Prerequisites: 1. An account exists 2. A valid unexpired one-time code has been issued to that account 3. Database can be accessed
Test Steps: 1. Member opens the password reset page 2. Member enters the one-time code 3. Member enters a new password meeting the complexity policy 4. Member clicks Reset Password 5. Member logs in with the new password
Test Data: 1. A valid unexpired one-time code 2. A new password meeting the complexity policy
Expected Results: The password is changed and the member can log in with it. The stored password is a hash, not readable text, and the one-time code no longer works.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-OTP-001, TestPasswordResetServlet.testSuccessfulResetHashesWithSecurityUtil

Test Case Task ID: #21
Test Scenario: Verify that an expired one-time code cannot be used to reset a password
Prerequisites: 1. A one-time code was issued and has since expired
Test Steps: 1. Member opens the password reset page 2. Member enters the expired one-time code 3. Member enters a new password 4. Member clicks Reset Password
Test Data: 1. An expired one-time code 2. A new password
Expected Results: The reset is refused with a message saying the code has expired, and the existing password still works.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-OTP-002, TestPasswordResetServlet.testExpiredOtpRejectsReset

Test Case Task ID: #22
Test Scenario: Verify that the password reset request does not reveal whether an email address is registered
Prerequisites: 1. One registered email address and one unregistered email address are available
Test Steps: 1. Guest opens the forgot password page 2. Guest submits the registered email address and notes the response 3. Guest submits the unregistered email address and notes the response
Test Data: 1. A registered email address 2. An unregistered email address
Expected Results: Both submissions return the same message. Nothing in the response indicates whether the address is registered, and no code is sent for the unregistered address.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-OTP-003, TestAuthApiServlet.forgotPasswordDoesNotEnumerate

Test Case Task ID: #23
Test Scenario: Verify that a one-time code cannot be guessed by repeated attempts
Prerequisites: 1. A valid one-time code has been issued to an account
Test Steps: 1. Member enters a wrong one-time code and submits 2. Member repeats this until the attempt limit is reached 3. Member enters the correct one-time code and submits
Test Data: 1. Wrong codes repeated up to the attempt limit 2. The correct one-time code
Expected Results: The code is invalidated once the attempt limit is reached, so even the correct code is refused and a new code must be requested.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-OTP-004, OtpStoreTest.attemptLimitInvalidatesOtp

Test Case Task ID: #24
Test Scenario: Verify that a member can turn on two-factor authentication
Prerequisites: 1. Member is logged in 2. Member has scanned the setup code into an authenticator app
Test Steps: 1. Member opens the security settings page 2. Member starts two-factor setup 3. Member enters the six digit code shown by the authenticator app 4. Member clicks Confirm
Test Data: 1. The current six digit code from the authenticator app
Expected Results: Two-factor authentication is turned on for the account and the setting is shown as enabled.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-OTP-005, TestTwoFactorServlet.testConfirmValidOtpEnablesTwoFactor

Test Case Task ID: #25
Test Scenario: Verify that a wrong two-factor code at login does not grant access
Prerequisites: 1. An account has two-factor authentication enabled 2. The correct password has already been entered
Test Steps: 1. Member enters an incorrect six digit code at the two-factor prompt 2. Member clicks Verify
Test Data: 1. An incorrect six digit code
Expected Results: Access is refused, the member stays on the two-factor prompt, and no logged in session is created.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-OTP-006, TestTwoFactorApiServlet.verifyLoginWrongOtp

Platform: Session, Access Control and Security

Test Case Task ID: #26
Test Scenario: Verify that a visitor with no session is denied every role restricted page
Prerequisites: 1. No user is logged in
Test Steps: 1. Guest enters the address of the admin dashboard and loads the page 2. Guest enters the address of the seller dashboard and loads the page 3. Guest enters the address of the watchlist page and loads the page
Test Data: 1. The addresses of the admin, seller and watchlist pages
Expected Results: Every page is refused and the guest is sent to the login page. No page grants access by default.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEC-001, TestTwoFactorServlet.testNullSessionDeniesAllAccess

Test Case Task ID: #27
Test Scenario: Verify that a buyer cannot reach any admin page
Prerequisites: 1. Buyer is logged in with an ordinary member account
Test Steps: 1. Buyer enters the address of the admin dashboard 2. Buyer loads the page
Test Data: 1. The address of the admin dashboard
Expected Results: Access is refused before the page loads, and no admin content is shown.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEC-002, TestAdminFilter.TestBuyer

Test Case Task ID: #28
Test Scenario: Verify that an admin can reach the admin dashboard
Prerequisites: 1. Admin is logged in
Test Steps: 1. Admin opens the admin dashboard
Test Data: 1. An account holding the admin role
Expected Results: The admin dashboard loads and its content is displayed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEC-003, TestAdminFilter.TestAdmin

Test Case Task ID: #29
Test Scenario: Verify that a session cannot be reused after logging out
Prerequisites: 1. Member is logged in and has a protected page open
Test Steps: 1. Member clicks Logout 2. Member presses the browser back button to return to the protected page 3. Member refreshes the page
Test Data: 1. A protected page opened before logout
Expected Results: The page is no longer accessible and the member is sent to the login page.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEC-004, TestLogoutServlet.testProtectedPageBlocksRequestAfterSessionInvalidation

Test Case Task ID: #30
Test Scenario: Verify that every response carries the browser security headers
Prerequisites: 1. The site is running
Test Steps: 1. Tester opens any page with the browser developer tools open 2. Tester inspects the response headers on the network tab
Test Data: 1. Any page request
Expected Results: The response carries the content security policy, content type options and frame options headers.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEC-005, TestSecurityFilter.securityHeaders

Test Case Task ID: #31
Test Scenario: Verify that personal data is encrypted at rest and that identical values do not produce identical stored values
Prerequisites: 1. Two member accounts exist 2. Database can be accessed
Test Steps: 1. Member A saves a phone number on the profile page 2. Member B saves the same phone number on their profile page 3. Tester reads both stored phone values from the database
Test Data: 1. One phone number saved on two different accounts
Expected Results: Neither stored value is readable text, and the two stored values differ from each other even though the entered number was the same.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEC-006, SecurityUtilTest.encryptUsesRandomlv

Test Case Task ID: #32
Test Scenario: Verify that text entered by a user cannot run as script when it is displayed
Prerequisites: 1. Member is logged in
Test Steps: 1. Member enters a script tag into a free text field such as a question or description 2. Member saves the entry 3. Member opens the page where that text is displayed
Test Data: 1. The text <script>alert(1)</script>
Expected Results: The text is shown as plain characters on the page and no script runs.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEC-007, SecurityUtilTest.sanitizeXss

Buyer: Bidding on Ascending, Dutch and Buy It Now Auctions

Test Case Task ID: #33
Test Scenario: Verify that a buyer can place a valid bid on an ascending auction
Prerequisites: 1. Buyer is logged in 2. A live ascending auction exists 3. Buyer does not own the auction
Test Steps: 1. Buyer opens the auction detail page 2. Buyer enters an amount above the current price 3. Buyer clicks Place Bid
Test Data: 1. An amount higher than the current price
Expected Results: The bid is accepted, the current price updates to the new amount, and a confirmation message is shown.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BID-001, TestPlaceBidServlet.successfulBid

Test Case Task ID: #34
Test Scenario: Verify that a seller cannot bid on their own listing
Prerequisites: 1. Seller is logged in 2. The seller owns a live auction
Test Steps: 1. Seller opens their own auction detail page 2. Seller enters a valid bid amount 3. Seller clicks Place Bid
Test Data: 1. A valid bid amount on the seller's own auction
Expected Results: The bid is refused with a message explaining a seller cannot bid on their own listing, and the price is unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BID-002, TestPlaceBidServlet.selfBidRejected

Test Case Task ID: #35
Test Scenario: Verify that a bid equal to the current price is refused
Prerequisites: 1. Buyer is logged in 2. A live ascending auction has a known current price
Test Steps: 1. Buyer opens the auction detail page 2. Buyer enters an amount exactly equal to the current price 3. Buyer clicks Place Bid
Test Data: 1. An amount equal to the current price
Expected Results: The bid is refused because the price must increase, and the current price is unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BID-003, TestPlaceBidServlet.equalBidRejected

Test Case Task ID: #36
Test Scenario: Verify that a negative bid amount is refused
Prerequisites: 1. Buyer is logged in 2. A live ascending auction exists
Test Steps: 1. Buyer opens the auction detail page 2. Buyer enters a negative amount 3. Buyer clicks Place Bid
Test Data: 1. The amount -50
Expected Results: A validation message is shown, the bid is not recorded, and the current price is unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BID-004, TestPlaceBidServlet.negativeBidAmount

Test Case Task ID: #37
Test Scenario: Verify that a bid on an auction that has already ended is refused
Prerequisites: 1. Buyer is logged in 2. An auction whose end time has passed exists
Test Steps: 1. Buyer opens the ended auction detail page 2. Buyer enters a valid amount 3. Buyer clicks Place Bid
Test Data: 1. A valid bid amount on an ended auction
Expected Results: The bid is refused with a message saying the auction is closed, and no bid is recorded.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BID-005, TestPlaceBidServlet.closedAuction

Test Case Task ID: #38
Test Scenario: Verify that the displayed price of a Dutch auction falls over time and never goes below the floor price
Prerequisites: 1. A Dutch auction exists with a start price, a floor price, a start time and an end time
Test Steps: 1. Buyer opens the Dutch auction detail page at the start time and notes the price 2. Buyer reopens the page halfway through the auction and notes the price 3. Buyer reopens the page at the end time and notes the price
Test Data: 1. A Dutch auction with a start price and a lower floor price
Expected Results: The price equals the start price at the start, has fallen to the midpoint value halfway through, and never drops below the floor price.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BID-006, DutchClockTest.listedPriceFollowsTheClock

Test Case Task ID: #39
Test Scenario: Verify that a bid below the required increment is refused and the minimum is shown
Prerequisites: 1. Buyer is logged in 2. A live auction exists with a known current price and increment
Test Steps: 1. Buyer opens the auction detail page 2. Buyer enters an amount below the required minimum 3. Buyer clicks Place Bid
Test Data: 1. An amount below the current price plus the increment
Expected Results: The bid is refused and the message states the minimum amount that would be accepted.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BID-007, TestBidApiServlet.bidTooLow

Test Case Task ID: #40
Test Scenario: Verify that a buyer cannot place bids faster than the cooldown allows
Prerequisites: 1. Buyer is logged in 2. Buyer has just placed a bid on the auction
Test Steps: 1. Buyer immediately enters a higher amount 2. Buyer clicks Place Bid before the cooldown has passed
Test Data: 1. A second bid placed inside the cooldown window
Expected Results: The second bid is refused with a message asking the buyer to wait, and only the first bid is recorded.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BID-008, BidDAORateLimitTest.secondBidInsideWindowRejected

Buyer: Blind (Sealed Bid) Auctions

Test Case Task ID: #41
Test Scenario: Verify that a rival bidder cannot see the leading bid while a blind auction is open
Prerequisites: 1. Buyer is logged in 2. A live blind auction has three sealed bids, the highest being 250 dollars 3. The buyer does not own the auction
Test Steps: 1. Buyer opens the blind auction detail page
Test Data: 1. A live blind auction with a leading sealed bid of 250 dollars
Expected Results: No current bid amount is displayed. The page marks the auction as sealed and shows only the number of bids received.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-001, TestBlindAuction.openBlindHidesTheStandingBidFromRivalBidders

Test Case Task ID: #42
Test Scenario: Verify that an unregistered visitor cannot see the leading bid of an open blind auction
Prerequisites: 1. A live blind auction with sealed bids exists 2. No user is logged in
Test Steps: 1. Guest opens the blind auction detail page
Test Data: 1. A live blind auction with a leading sealed bid of 250 dollars
Expected Results: No current bid amount is displayed, and the page does not treat the visitor as the owner.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-002, TestBlindAuction.openBlindHidesTheStandingBidFromAnonymousVisitors

Test Case Task ID: #43
Test Scenario: Verify that the seller can see the standing bid on their own blind auction
Prerequisites: 1. Seller is logged in 2. The seller owns a live blind auction with a leading sealed bid of 250 dollars
Test Steps: 1. Seller opens their own blind auction detail page
Test Data: 1. A live blind auction owned by the logged in seller
Expected Results: The standing bid of 250 dollars is displayed to the seller, so that the Declare Winner Early option can be used at that price.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-003, TestBlindAuction.theSellerSeesTheStandingBidOnTheirOwnListing

Test Case Task ID: #44
Test Scenario: Verify that the winning amount of a blind auction is revealed once it closes
Prerequisites: 1. Buyer is logged in 2. A blind auction has passed its end time
Test Steps: 1. Buyer opens the closed blind auction detail page
Test Data: 1. A closed blind auction whose winning bid was 250 dollars
Expected Results: The winning bid of 250 dollars is displayed and the auction is no longer marked as sealed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-004, TestBlindAuction.closedBlindRevealsTheWinningBid

Test Case Task ID: #45
Test Scenario: Verify that a bidder can see their own sealed bid but not anyone else's
Prerequisites: 1. Buyer is logged in 2. The buyer has submitted a sealed bid of 120 dollars 3. Another bidder holds the leading bid of 250 dollars
Test Steps: 1. Buyer opens the blind auction detail page
Test Data: 1. The buyer's own sealed bid of 120 dollars
Expected Results: The buyer's own bid of 120 dollars is shown, and no leading bid amount is displayed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-005, TestBlindAuction.aBidderSeesTheirOwnBidButNotTheOthers

Test Case Task ID: #46
Test Scenario: Verify that the bid history is empty while a blind auction is still open
Prerequisites: 1. A live blind auction has three sealed bids
Test Steps: 1. Buyer opens the blind auction detail page 2. Buyer opens the bid history panel
Test Data: 1. A live blind auction with three sealed bids
Expected Results: The bid history lists no entries. The total count of three is shown, but no amount or bidder appears.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-006, TestBlindAuction.openBlindBidHistoryReturnsNoRows

Test Case Task ID: #47
Test Scenario: Verify that the live price update on an open blind auction carries no amount
Prerequisites: 1. A live blind auction with a leading bid of 250 dollars is open in the browser 2. Another bidder places a sealed bid
Test Steps: 1. Buyer keeps the blind auction detail page open 2. A second bidder submits a sealed bid 3. Buyer watches the page update without reloading
Test Data: 1. A sealed bid submitted by another bidder while the page is open
Expected Results: The bid count on the page increases, and no bid amount is displayed at any point.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-007, AuctionEventPublisherBlindTest.openBlindSnapshotCarriesNoAmount

Test Case Task ID: #48
Test Scenario: Verify that no listing view reveals the leading bid of a live blind auction
Prerequisites: 1. A live blind auction exists with a starting price of 100 dollars and a leading sealed bid of 250 dollars
Test Steps: 1. Guest finds the blind auction in the search results and notes the price on the card 2. Guest applies a price filter that would only match the sealed amount 3. Guest checks the trending listings strip 4. Guest checks the featured listings strip
Test Data: 1. A blind auction with a starting price of 100 dollars and a sealed leading bid of 250 dollars
Expected Results: Every view shows the starting price of 100 dollars. The price filter does not single the listing out, so the sealed amount cannot be worked out from any listing view.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-008, TestBlindAuction.searchResolvesABlindListingToltsStartingPrice

Test Case Task ID: #49
Test Scenario: Verify that a buyer can submit a sealed bid and is told only that it was received
Prerequisites: 1. Buyer is logged in 2. A live blind auction exists 3. The buyer has not yet bid on it
Test Steps: 1. Buyer opens the blind auction detail page 2. Buyer enters a sealed bid amount 3. Buyer clicks Submit Sealed Bid
Test Data: 1. A sealed bid of 250 dollars
Expected Results: A confirmation states that the sealed bid was received. No price, ranking or comparison against other bids is shown.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-009, TestBlindAuction.aSealedBidIsAcknowledgedWithoutRevealingAnything

Test Case Task ID: #50
Test Scenario: Verify that a buyer cannot submit a second sealed bid on the same auction
Prerequisites: 1. Buyer is logged in 2. The buyer has already submitted a sealed bid on this auction
Test Steps: 1. Buyer opens the same blind auction detail page 2. Buyer enters a higher amount 3. Buyer clicks Submit Sealed Bid
Test Data: 1. A second sealed bid higher than the buyer's first
Expected Results: The submission is refused with the message that a sealed bid has already been submitted for this auction, and the original bid is unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-010, TestBlindAuction.aSecondSealedBidFromTheSameBuyerIsRefused

Test Case Task ID: #51
Test Scenario: Verify that the entry price of a blind auction is an inclusive minimum to the cent
Prerequisites: 1. A live blind auction has a starting price of 100 dollars 2. Two buyer accounts are available
Test Steps: 1. First buyer submits a sealed bid of exactly 100.00 dollars 2. Second buyer submits a sealed bid of 99.99 dollars
Test Data: 1. A sealed bid of 100.00 dollars 2. A sealed bid of 99.99 dollars
Expected Results: The bid of 100.00 dollars is accepted. The bid of 99.99 dollars is refused as too low and is not recorded.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-011, TestBlindAuction.aBidAtExactlyTheEntryPriceIsAccepted

Test Case Task ID: #52
Test Scenario: Verify that Buy It Now and Dutch acceptance cannot be used on a blind auction
Prerequisites: 1. Buyer is logged in 2. A live blind auction exists
Test Steps: 1. Buyer attempts a Buy It Now purchase on the blind auction 2. Buyer attempts to accept the current Dutch clock price on the blind auction
Test Data: 1. A Buy It Now request on a blind auction 2. A Dutch acceptance request on a blind auction
Expected Results: Both attempts are refused with a message that the action does not apply to this auction type, and nothing is purchased.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-012, TestBlindAuction.buyItNowDoesNotApplyToABlindAuction

Test Case Task ID: #53
Test Scenario: Verify that submitting a sealed bid sends no outbid or new bid notification
Prerequisites: 1. A live blind auction already has sealed bids from other buyers 2. Those buyers have notifications turned on
Test Steps: 1. A new buyer submits a higher sealed bid 2. Earlier bidders check their notifications 3. The seller checks their notifications
Test Data: 1. A new sealed bid that exceeds the existing highest
Expected Results: No outbid notification reaches the earlier bidders and no new bid notification reaches the seller, so no message discloses the standing amount.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-013, TestBlindAuction.aSealedBidNotifiesNoOutbidBidder

Test Case Task ID: #54
Test Scenario: Verify that the highest sealed bid wins when a blind auction closes and the order is raised for that amount
Prerequisites: 1. A blind auction has several sealed bids 2. The highest is 250 dollars from one buyer 3. The auction has passed its end time
Test Steps: 1. Tester waits for the auction to be finalised 2. Winning buyer opens their purchases page 3. Losing bidders check their notifications
Test Data: 1. Sealed bids where the highest is 250 dollars
Expected Results: The buyer who bid 250 dollars is the winner, one order is raised for exactly 250.00 dollars, the winner is notified that they won, and every other bidder is notified that they lost.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-014, TestBlindAuction.theHighestSealedBidWins

Test Case Task ID: #55
Test Scenario: Verify that watchlisting a live blind auction does not reveal its leading bid
Prerequisites: 1. Buyer is logged in 2. A live blind auction has a starting price of 100 dollars and a leading sealed bid of 250 dollars
Test Steps: 1. Buyer adds the blind auction to their watchlist 2. Buyer opens the watchlist page 3. Buyer reads the price shown on the watchlist entry
Test Data: 1. A live blind auction on the buyer's watchlist
Expected Results: The watchlist shows the starting price of 100 dollars. The sealed leading bid of 250 dollars does not appear.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-015, TestBlindAuction.watchlistDoesNotLeakTheSealedBid

Test Case Task ID: #56
Test Scenario: Verify that a public seller profile does not reveal the leading bid of a live blind auction
Prerequisites: 1. A seller has a live blind listing with a starting price of 100 dollars and a leading sealed bid of 250 dollars 2. No user is logged in
Test Steps: 1. Guest opens the seller's public profile page 2. Guest reads the price shown against the blind listing
Test Data: 1. A seller profile showing a live blind listing
Expected Results: The profile shows the starting price of 100 dollars. The sealed leading bid does not appear to a visitor with no session.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-016, TestBlindAuction.sellerProfileDoesNotLeakTheSealedBid

Test Case Task ID: #57
Test Scenario: Verify that the bid history of a live blind auction is empty on every page that displays it
Prerequisites: 1. A live blind auction has sealed bids
Test Steps: 1. Guest opens the auction detail page and checks the bid history panel 2. Guest opens the standalone bid history page for the same auction directly by its address
Test Data: 1. A live blind auction with sealed bids
Expected Results: Neither page lists any bid amount. Entering the bid history address directly does not disclose the sealed bids.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-017, TestBlindAuction.bidHistoryHidesLiveSealedBidsForEveryCaller

Test Case Task ID: #58
Test Scenario: Verify that an ordinary ascending bid on a blind auction is refused outright
Prerequisites: 1. Buyer is logged in 2. A live blind auction exists 3. The leading sealed amount is not known to the buyer
Test Steps: 1. Buyer submits an ordinary bid of 250 dollars through the standard bidding page instead of the sealed bid form 2. Buyer repeats with several different amounts and compares the messages
Test Data: 1. Ordinary bids of several different amounts on a blind auction
Expected Results: Every attempt is refused with the same message that the action does not apply to this auction type. No bid is recorded, and the wording never differs according to the amount, so it cannot be used to work out the sealed bid.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-018, TestBlindAuction.ascendingBidOnABlindAuctionIsRefused

Test Case Task ID: #59
Test Scenario: Verify that the search result count cannot be used to work out a sealed bid
Prerequisites: 1. A live blind auction has a starting price of 100 dollars and a leading sealed bid of 250 dollars
Test Steps: 1. Guest searches with a price range that excludes 100 dollars but includes 250 dollars 2. Guest notes the result count shown 3. Guest narrows the price range repeatedly and watches the count
Test Data: 1. A series of narrowing minimum and maximum price filters
Expected Results: The result count matches the listings actually shown on the page at every step. The blind auction is never counted against its sealed amount, so narrowing the range does not locate it.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-019, TestBlindAuction.thePriceFilteredCountCannotProbeTheSealedBid

Test Case Task ID: #60
Test Scenario: Verify that a blind auction reveals its winning bid once it has ended
Prerequisites: 1. A blind auction with sealed bids has passed its end time 2. The auction is on a buyer's watchlist
Test Steps: 1. Buyer opens the watchlist page after the auction has ended 2. Buyer opens the auction detail page 3. Buyer opens the bid history panel
Test Data: 1. A blind auction that has passed its end time
Expected Results: The winning amount is shown on the watchlist, on the detail page and in the bid history. The hiding applies only while the auction is running.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-020, TestBlindAuction.aConcludedBlindAuctionIsNotHidden

Test Case Task ID: #61
Test Scenario: Verify that an auto-bid cannot be set on a blind auction
Prerequisites: 1. Buyer is logged in 2. A live blind auction exists
Test Steps: 1. Buyer opens the blind auction detail page 2. Buyer attempts to set an auto-bid ceiling of 500 dollars
Test Data: 1. An auto-bid ceiling of 500 dollars on a blind auction
Expected Results: The request is refused with a message explaining that a sealed auction takes one hidden bid instead, and no auto-bid is stored.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-021, TestBlindAuction.settingAnAutoBidIsRefused

Test Case Task ID: #62
Test Scenario: Verify that an auto-bid stored on a blind auction before the restriction can still be cancelled
Prerequisites: 1. Buyer is logged in 2. An auto-bid row exists against a blind auction from before the restriction
Test Steps: 1. Buyer cancels the auto-bid on that blind auction
Test Data: 1. An existing auto-bid row on a blind auction
Expected Results: The cancellation succeeds and the stored auto-bid is removed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-022, TestBlindAuction.cancellingIsStillAllowed

Test Case Task ID: #63
Test Scenario: Verify that no auto-bid is reported for a blind auction even if one is still stored
Prerequisites: 1. Buyer is logged in 2. An auto-bid row still exists against a blind auction
Test Steps: 1. Buyer opens the blind auction detail page 2. Buyer checks the auto-bid area
Test Data: 1. A blind auction with a leftover auto-bid row
Expected Results: The page reports that no auto-bid is set, and the stored value is not displayed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-023, TestBlindAuction.readingOneBackReportsNone

Test Case Task ID: #64
Test Scenario: Verify that a blind auction page does not display an Auto-Bid Active panel
Prerequisites: 1. Buyer is logged in 2. The buyer has a leftover auto-bid of 900 dollars against a live blind auction
Test Steps: 1. Buyer opens the blind auction detail page
Test Data: 1. A leftover auto-bid of 900 dollars on a live blind auction
Expected Results: No Auto-Bid Active panel is shown and the 900 dollar ceiling is not displayed, so the buyer is not promised a defence that will never run.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-024, TestBlindAuction.openBlindDoesNotEchoAnAutoBid

Test Case Task ID: #65
Test Scenario: Verify that the blind auction restriction has not stopped auto-bidding on ordinary auctions
Prerequisites: 1. Buyer is logged in 2. A live ascending auction exists
Test Steps: 1. Buyer opens the ascending auction detail page 2. Buyer sets an auto-bid ceiling of 500 dollars 3. Buyer clicks Save
Test Data: 1. An auto-bid ceiling of 500 dollars on an ascending auction
Expected Results: The auto-bid is accepted and stored as before, and the Auto-Bid Active panel is shown.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BLD-025, TestBlindAuction.anAscendingAuctionStillAcceptsOne

Buyer: Auto-Bid and Proxy Bidding

Test Case Task ID: #66
Test Scenario: Verify that a buyer can set an auto-bid ceiling on a live auction
Prerequisites: 1. Buyer is logged in 2. A live ascending auction exists 3. The buyer does not own the auction
Test Steps: 1. Buyer opens the auction detail page 2. Buyer enters a maximum amount in the auto-bid field 3. Buyer clicks Set Auto-Bid
Test Data: 1. A maximum auto-bid amount above the current price
Expected Results: The auto-bid is saved, a confirmation is shown, and the stored ceiling is not readable in plain form in the database.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-AUT-001, TestSetAutoBidServlet.successfulSet

Test Case Task ID: #67
Test Scenario: Verify that when two buyers hold auto-bids the higher ceiling takes the lead
Prerequisites: 1. Two buyers have set auto-bids on the same live auction with different maximum amounts
Test Steps: 1. A new bid is placed that triggers both auto-bids 2. Tester opens the auction detail page and reads the leading bid
Test Data: 1. Two auto-bid ceilings of different amounts on one auction
Expected Results: The buyer with the higher ceiling leads, at one increment above the lower ceiling rather than at their own maximum.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-AUT-002, TestSetAutoBidServlet.competingHigherMaxWins

Test Case Task ID: #68
Test Scenario: Verify that two identical auto-bid ceilings are settled in favour of whoever set theirs first
Prerequisites: 1. Two buyers have set auto-bids with exactly the same maximum amount on the same auction 2. The order in which they were set is known
Test Steps: 1. A new bid is placed that triggers both auto-bids 2. Tester opens the auction detail page and reads the leading bidder
Test Data: 1. Two identical auto-bid ceilings set at different times
Expected Results: The buyer who set their auto-bid first holds the lead. The result is the same on every repeat, not arbitrary.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-AUT-003, TestSetAutoBidServlet.equalMaxFifoTiebreak

Test Case Task ID: #69
Test Scenario: Verify that an auto-bid that can no longer be read is treated as absent rather than causing an error
Prerequisites: 1. An auto-bid was stored under an encryption key that has since been rotated 2. Database can be accessed
Test Steps: 1. Buyer opens the auction detail page carrying that auto-bid 2. A new bid is placed on the auction
Test Data: 1. An auto-bid stored under a superseded encryption key
Expected Results: The page reports no auto-bid set, the auction continues to work, and no bid is placed at an incorrect amount.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-AUT-004, AutoBidDAOStaleKeyTest.undecryptableRowReadsAsAbsent

Buyer: Watchlist, Questions, Ratings and Reports

Test Case Task ID: #70
Test Scenario: Verify that a buyer can add a listing to their watchlist
Prerequisites: 1. Buyer is logged in 2. The auction is not already on the watchlist
Test Steps: 1. Buyer opens an auction detail page 2. Buyer clicks Add to Watchlist 3. Buyer opens the watchlist page
Test Data: 1. An existing live auction
Expected Results: A confirmation is shown and the auction appears on the buyer's watchlist.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BUY-001, TestWatchlistServlet.addSuccess

Test Case Task ID: #71
Test Scenario: Verify that an unregistered visitor cannot view a watchlist
Prerequisites: 1. No user is logged in
Test Steps: 1. Guest enters the address of the watchlist page 2. Guest loads the page
Test Data: 1. The address of the watchlist page
Expected Results: Access is refused and no watchlist entries are shown.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BUY-002, TestWatchlistApiServlet.getForbiddenForGuest

Test Case Task ID: #72
Test Scenario: Verify that a buyer can ask the seller a question and that the text is cleaned before it is shown
Prerequisites: 1. Buyer is logged in 2. The auction does not belong to the buyer
Test Steps: 1. Buyer opens the auction detail page 2. Buyer types a question containing HTML markup 3. Buyer clicks Submit Question 4. Seller checks their notifications
Test Data: 1. Question text containing an HTML tag
Expected Results: The question is posted with the markup shown as plain characters, and the seller receives a notification.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BUY-003, TestAuctionQuestionServlet.testAskSuccessSanitized

Test Case Task ID: #73
Test Scenario: Verify that only the owning seller can answer a question on a listing
Prerequisites: 1. A seller is logged in who does not own the auction 2. An unanswered question exists on that auction
Test Steps: 1. Seller opens the auction detail page 2. Seller attempts to submit a reply to the question
Test Data: 1. A reply to a question on another seller's listing
Expected Results: The reply is refused and no answer is stored against the question.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BUY-004, TestAuctionQuestionServlet.testReplyWrongSeller403

Test Case Task ID: #74
Test Scenario: Verify that only the winning buyer can rate the seller
Prerequisites: 1. Buyer is logged in 2. The auction has finished 3. The logged in buyer did not win it
Test Steps: 1. Buyer opens the finished auction page 2. Buyer attempts to submit a rating of 5 for the seller
Test Data: 1. A rating score of 5 from a buyer who did not win
Expected Results: The rating is refused and no rating is recorded against the seller.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BUY-005, TestRateSellerServlet.buyerNotWinner

Test Case Task ID: #75
Test Scenario: Verify that the rating scale accepts its highest value
Prerequisites: 1. Buyer is logged in 2. The buyer won the auction 3. The buyer has not yet rated the seller
Test Steps: 1. Buyer opens the finished auction page 2. Buyer selects the maximum score of 5 3. Buyer clicks Submit Rating
Test Data: 1. A rating score of 5
Expected Results: The rating is accepted and appears on the seller's profile.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BUY-006, TestRateSellerServlet.scoreBoundaryMax

Test Case Task ID: #76
Test Scenario: Verify that a buyer cannot report the same listing twice
Prerequisites: 1. Buyer is logged in 2. The buyer has already submitted a report on this listing that is still open
Test Steps: 1. Buyer opens the same listing 2. Buyer submits a second report
Test Data: 1. A second report on a listing already reported by this buyer
Expected Results: The second report is refused with a message saying the listing has already been reported, and only one report exists.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-BUY-007, TestBuyerReportServlet.duplicateReport

Buyer: Orders, Payment, Shipping and Refunds

Test Case Task ID: #77
Test Scenario: Verify that a winning buyer can pay for an order with a saved payment method
Prerequisites: 1. Buyer is logged in 2. The buyer has an unpaid order 3. The buyer has a saved payment method
Test Steps: 1. Buyer opens the purchases page 2. Buyer selects the unpaid order 3. Buyer chooses their saved payment method 4. Buyer clicks Pay
Test Data: 1. An unpaid order 2. A saved payment method belonging to the buyer
Expected Results: The order is marked as paid, and both the buyer and the seller receive a notification.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ORD-001, TestOrderApiServlet.paySuccess

Test Case Task ID: #78
Test Scenario: Verify that a buyer cannot pay using another user's saved payment method
Prerequisites: 1. Buyer is logged in with an unpaid order 2. A payment method belonging to a different user is known
Test Steps: 1. Buyer opens the unpaid order 2. Buyer submits payment referencing another user's saved payment method
Test Data: 1. A payment method identifier belonging to a different user
Expected Results: The payment is refused and the order remains unpaid.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ORD-002, TestOrderApiServlet.payRejectsForeignMethod

Test Case Task ID: #79
Test Scenario: Verify that shipping cannot be advanced before the order is paid
Prerequisites: 1. Seller is logged in 2. An order on the seller's listing is still unpaid
Test Steps: 1. Seller opens the sales page 2. Seller selects the unpaid order 3. Seller attempts to mark it as despatched
Test Data: 1. An unpaid order
Expected Results: The change is refused and the shipping stage stays as it was.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ORD-003, TestOrderApiServlet.shippingRejectsUnpaidOrder

Test Case Task ID: #80
Test Scenario: Verify that a refund request must carry a reason of the minimum length
Prerequisites: 1. Buyer is logged in 2. The buyer has a paid order
Test Steps: 1. Buyer opens the paid order 2. Buyer clicks Request Refund 3. Buyer enters a reason shorter than the minimum length 4. Buyer clicks Submit
Test Data: 1. A refund reason shorter than the minimum permitted length
Expected Results: A validation message is shown and no refund request is raised.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ORD-004, TestOrderApiServlet.refundReasonTooShort

Test Case Task ID: #81
Test Scenario: Verify that an unpaid order past its payment deadline is cancelled automatically
Prerequisites: 1. An unpaid order exists that is older than the configured payment deadline 2. A separate paid order also exists
Test Steps: 1. Tester waits for the scheduled payment timeout to run 2. Tester opens the order list and checks both orders
Test Data: 1. One unpaid order past the deadline 2. One paid order
Expected Results: The overdue unpaid order is cancelled. The paid order is untouched and the related auction records are unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ORD-005, OrderDAOPaymentTimeoutTest.overdueOrderIsCancelled

Test Case Task ID: #82
Test Scenario: Verify that a saved card number cannot be changed through the edit form
Prerequisites: 1. Buyer is logged in 2. The buyer has a saved card
Test Steps: 1. Buyer opens the payment methods page 2. Buyer opens the saved card for editing 3. Buyer attempts to submit a different card number along with a new expiry date
Test Data: 1. A different card number 2. A new expiry date
Expected Results: The card number is unchanged. Only the cardholder name and expiry date are updated.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ORD-006, AccountApiPaymentMethodUpdateTest.updateRefusesPanChange

Seller: Listings, Maintenance and Analytics

Test Case Task ID: #83
Test Scenario: Verify that a seller can publish a listing with images and tags
Prerequisites: 1. Seller is logged in with selling turned on 2. Database can be accessed
Test Steps: 1. Seller opens the create listing page 2. Seller fills in the listing details 3. Seller uploads one or more images 4. Seller adds tags 5. Seller clicks Publish
Test Data: 1. A complete set of listing details 2. One or more permitted image files 3. One or more tags
Expected Results: The listing is published and appears in search results with its images and tags attached.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEL-001, TestCreateAuctionServlet.TestValidTags

Test Case Task ID: #84
Test Scenario: Verify that a member without selling turned on cannot create a listing
Prerequisites: 1. A member is logged in who has not turned selling on
Test Steps: 1. Member enters the address of the create listing page 2. Member fills in the form and clicks Publish
Test Data: 1. A complete listing form submitted by a buyer only account
Expected Results: The request is refused and no listing is created.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEL-002, TestCreateAuctionServlet.TestNotSeller

Test Case Task ID: #85
Test Scenario: Verify that only permitted image types can be uploaded to a listing
Prerequisites: 1. Seller is logged in with selling turned on
Test Steps: 1. Seller opens the create listing page 2. Seller selects a file with a disallowed extension 3. Seller clicks Publish
Test Data: 1. A file with a disallowed extension
Expected Results: The upload is refused with a file type message, and no file is saved on the server.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEL-003, TestCreateAuctionServlet.testInvalidExtension

Test Case Task ID: #86
Test Scenario: Verify that an image larger than the size limit is refused
Prerequisites: 1. Seller is logged in with selling turned on
Test Steps: 1. Seller opens the create listing page 2. Seller selects an image larger than the permitted size 3. Seller clicks Publish
Test Data: 1. An image file exceeding the size limit
Expected Results: The upload is refused with a size message and the listing is not published.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEL-004, TestCreateAuctionServlet.testFileTooLarge

Test Case Task ID: #87
Test Scenario: Verify that a listing that already has bids cannot be edited
Prerequisites: 1. Seller is logged in 2. The seller owns a live auction that has at least one bid
Test Steps: 1. Seller opens their listing 2. Seller clicks Edit 3. Seller changes a field and clicks Save
Test Data: 1. An edit to a listing that already has bids
Expected Results: The edit is refused with a message explaining that bidders bid on the original terms, and the listing is unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEL-005, TestEditAuctionServlet.hasBidsBlocksEdit

Test Case Task ID: #88
Test Scenario: Verify that a bid arriving while the edit form is open still blocks the edit
Prerequisites: 1. Seller is logged in 2. The seller owns a live auction with no bids yet
Test Steps: 1. Seller opens the edit form while the listing has no bids 2. A buyer places a bid on that listing in another browser 3. Seller changes a field and clicks Save
Test Data: 1. A bid placed after the edit form was opened but before it was saved
Expected Results: The edit is refused because the check is repeated at save time, and the listing is unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEL-006, TestEditAuctionServlet.toctouBidsPlacedAfterGet

Test Case Task ID: #89
Test Scenario: Verify that removing the last unit of stock ends the listing and tells the bidders
Prerequisites: 1. Seller is logged in 2. The seller owns a listing with a quantity of 1 3. The listing has existing bidders
Test Steps: 1. Seller opens the listing 2. Seller removes the remaining unit and enters a reason 3. Seller confirms 4. Bidders check their notifications
Test Data: 1. A listing with a quantity of 1 2. A removal reason
Expected Results: The listing is ended, the reason is recorded against it, and every bidder receives a notification.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-SEL-007, TestSellerListingMaintenance.lastUnitEndsListingAndNotifies

Admin: Console and Database Backup

Test Case Task ID: #90
Test Scenario: Verify that an admin can suspend an active member account
Prerequisites: 1. Admin is logged in 2. Database can be accessed 3. At least one active member account exists
Test Steps: 1. Admin opens the user management page 2. Admin clicks Suspend next to an active member account
Test Data: 1. An existing active member account
Expected Results: The status of the account becomes SUSPENDED.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ADM-001, TestAdminManageUserServlet.TestSuspend

Test Case Task ID: #91
Test Scenario: Verify that an admin cannot moderate their own account
Prerequisites: Admin is logged in
Test Steps: - Admin opens the user management page - Admin attempts to suspend their own account
Test Data: The logged in admin's own account
Expected Results: The action is refused and the admin's own account status is unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ADM-002, TestAdminManageUserServlet.TestSelfAction

Test Case Task ID: #92
Test Scenario: Verify that one admin cannot change another admin's account status
Prerequisites: - Admin is logged in - A second admin account exists
Test Steps: - Admin opens the user management page - Admin attempts to change the status of the other admin account
Test Data: A second admin account
Expected Results: The action is refused, because admin accounts cannot be moderated from this page.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ADM-003, TestAdminManageUserServlet.TestUnbanAdminTarget

Test Case Task ID: #93
Test Scenario: Verify that a buyer cannot open the report moderation queue
Prerequisites: 1. Buyer is logged in with an ordinary member account
Test Steps: 1. Buyer enters the address of the admin reports page 2. Buyer loads the page
Test Data: 1. The address of the admin reports page
Expected Results: Access is refused and no report data is shown.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ADM-004, TestAdminReportServlet.testGetNonAdmin

Test Case Task ID: #94
Test Scenario: Verify that a category still used by live auctions cannot be deleted
Prerequisites: 1. Admin is logged in 2. A category is referenced by at least one live auction
Test Steps: 1. Admin opens the category management page 2. Admin clicks Delete next to that category
Test Data: 1. A category referenced by live auctions
Expected Results: The deletion is refused with a message explaining the category is still in use, and the category remains.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ADM-005, TestAdminCategoriesServlet.testDeleteRestrictedByAuctions

Test Case Task ID: #95
Test Scenario: Verify that a restore file containing destructive statements is rejected
Prerequisites: 1. Admin is logged in 2. A backup file containing DROP or DELETE statements is prepared
Test Steps: 1. Admin opens the database backup and restore page 2. Admin uploads the prepared file 3. Admin clicks Restore
Test Data: 1. A backup file containing a DROP statement
Expected Results: The file is rejected before anything runs, and the database is unchanged.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ADM-006, DatabaseBackupUtilTest.rejectDestructive

Test Case Task ID: #96
Test Scenario: Verify that a restore file is accepted when a semicolon appears inside a genuine data value
Prerequisites: 1. Admin is logged in 2. A valid backup file is prepared where one inserted value contains a semicolon
Test Steps: 1. Admin opens the database backup and restore page 2. Admin uploads the prepared file 3. Admin clicks Restore
Test Data: 1. A backup file whose inserted value contains a semicolon
Expected Results: The restore completes and the value is stored with its semicolon intact, so a legitimate value is not mistaken for a second statement.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ADM-007, DatabaseBackupUtilTest.allowsSemicolonInAValue

Platform: Recommendation Engine

Test Case Task ID: #97
Test Scenario: Verify that a signed in buyer receives personalised recommendations
Prerequisites: 1. Buyer is logged in 2. The buyer has browsing or bidding history
Test Steps: 1. Buyer opens the home page 2. Buyer reads the recommendations strip
Test Data: 1. A buyer account with interaction history
Expected Results: The strip shows recommendations related to the buyer's history and is labelled as personalised.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REC-001, TestRecommendationApiServlet.personalisedForBuyer

Test Case Task ID: #98
Test Scenario: Verify that an unregistered visitor still receives useful recommendations
Prerequisites: 1. No user is logged in
Test Steps: 1. Guest opens the home page 2. Guest reads the recommendations strip
Test Data: 1. A home page visit with no session
Expected Results: The strip shows trending listings and is not labelled as personalised.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REC-002, TestRecommendationApiServlet.trendingForAnonymous

Test Case Task ID: #99
Test Scenario: Verify that a newly registered user is not shown an empty recommendations strip
Prerequisites: 1. A newly registered account exists with no browsing or bidding history
Test Steps: 1. New member logs in 2. New member opens the home page 3. New member reads the recommendations strip
Test Data: 1. An account with no interaction history
Expected Results: The strip is filled with trending listings rather than left empty.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REC-003, TestRecommendationPipeline.coldStartFallsThroughToTrending

Test Case Task ID: #100
Test Scenario: Verify that a single category cannot fill the whole recommendations strip
Prerequisites: 1. Buyer is logged in 2. The buyer's history is concentrated in one category
Test Steps: 1. Buyer opens the home page 2. Buyer counts the recommendations belonging to each category
Test Data: 1. A buyer history concentrated in one category
Expected Results: No category exceeds the configured limit, and the strip still holds a full set of recommendations.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REC-004, TestRecommendationPipeline.categoryCapLimitsASingleCategory

Test Case Task ID: #101
Test Scenario: Verify that recent activity influences recommendations more than older activity
Prerequisites: 1. Buyer is logged in 2. The buyer has interactions of clearly different ages
Test Steps: 1. Buyer opens the home page 2. Buyer expands the why this panel on the recommendations and reads the scores
Test Data: 1. Interactions from today and from several weeks ago
Expected Results: Recent interactions carry more weight than older ones, and an interaction dated in the future does not increase a score.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REC-005, TestRecommendationPipeline.recencyMultiplierFadesWithAge

Test Case Task ID: #102
Test Scenario: Verify that the recommendation explanation never identifies a single other user
Prerequisites: 1. Buyer is logged in 2. A recommended item has been interacted with by exactly one other user
Test Steps: 1. Buyer opens the home page 2. Buyer expands the why this panel on that recommendation
Test Data: 1. A recommended item with exactly one other interacting user
Expected Results: The explanation is worded generally and contains no name, masked or otherwise, that would identify that one user.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-REC-006, TestRecommendationProvenance.withholdsMaskedNameForLoneClicker

Platform: Notifications and Preferences

Test Case Task ID: #103
Test Scenario: Verify that the leading bidder is told when a higher bid displaces them
Prerequisites: 1. An ascending auction has a leading bidder 2. A second buyer is logged in
Test Steps: 1. Second buyer places a higher bid 2. Displaced bidder checks their notifications 3. A third buyer who never bid checks their notifications
Test Data: 1. A bid higher than the current leading bid
Expected Results: The displaced bidder receives exactly one outbid notification. No other user receives it.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-NOT-001, BidApiNotificationTest.plainBidNotifiesThePreviousLeader

Test Case Task ID: #104
Test Scenario: Verify that every losing bidder is told the auction ended and the winner is not
Prerequisites: 1. An auction with several bidders has passed its end time
Test Steps: 1. Tester waits for the auction to be finalised 2. Each losing bidder checks their notifications 3. The winning bidder checks their notifications
Test Data: 1. An ended auction with one winner and several losing bidders
Expected Results: Each losing bidder receives one notification that they did not win. The winner receives the won notification instead and no losing notification.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-NOT-002, NotificationServiceLostTest.everyLoserExceptTheWinner

Test Case Task ID: #105
Test Scenario: Verify that a losing bidder notification does not reveal who won
Prerequisites: 1. An auction with several bidders has been finalised
Test Steps: 1. Losing bidder opens their notifications 2. Losing bidder reads the full text of the losing notification
Test Data: 1. A losing bidder notification
Expected Results: The message states that the auction ended without naming or otherwise identifying the winning bidder.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-NOT-003, NotificationServiceLostTest.bodyNeverNamesTheWinner

Test Case Task ID: #106
Test Scenario: Verify that rapid bidding does not flood the seller with notifications
Prerequisites: 1. Seller is logged in 2. The seller owns a live auction
Test Steps: 1. Several buyers place bids in quick succession on that auction 2. Seller opens their notifications
Test Data: 1. Several bids placed within a short window
Expected Results: The seller receives a single combined notification carrying the current price rather than one message per bid.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-NOT-004, NotificationServiceSellerAlertsTest.aBiddingWarCollapsesIntoOneMessage

Test Case Task ID: #107
Test Scenario: Verify that a member can switch off every optional notification
Prerequisites: 1. Member is logged in
Test Steps: 1. Member opens the notification preferences page 2. Member turns off every optional notification 3. Member clicks Save 4. An event that would normally raise an optional notification occurs
Test Data: 1. All optional notification toggles set to off
Expected Results: The preferences are saved, and no further optional notification is delivered to that member.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-NOT-005, TestUpdatePreferenceServlet.testAllFalse

Platform: Telegram Notification Integration

Test Case Task ID: #108
Test Scenario: Verify that a member can start linking their Telegram account
Prerequisites: 1. Member is logged in 2. The Telegram bot is configured
Test Steps: 1. Member opens the notification settings page 2. Member clicks Link Telegram
Test Data: 1. A member account with no Telegram link
Expected Results: The page shows both a link that opens the bot directly and a one-time code that can be sent to the bot by hand.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-TEL-001, TelegramApiServletTest.linkStartMintsBothPaths

Test Case Task ID: #109
Test Scenario: Verify that the Telegram webhook only accepts calls carrying the correct secret token
Prerequisites: 1. The Telegram bot is configured with a secret token
Test Steps: 1. Tester sends a webhook call carrying a wrong secret token 2. Tester repeats with a token that is a leading portion of the correct one
Test Data: 1. A wrong secret token 2. A token matching only the first characters of the correct one
Expected Results: Both calls are rejected and nothing is changed by either of them.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-TEL-002, TelegramWebhookServletTest.wrongSecretIsRejected

Test Case Task ID: #110
Test Scenario: Verify that link codes cannot be guessed by repeated attempts through the bot
Prerequisites: 1. The Telegram bot is configured 2. A chat is available for testing
Test Steps: 1. Tester sends a wrong link code to the bot 2. Tester repeats until the attempt limit is reached 3. Tester sends one more code
Test Data: 1. Wrong link codes repeated up to the attempt limit
Expected Results: The chat is blocked from further attempts once the limit is reached.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-TEL-003, TelegramWebhookServletTest.bruteForcelIsBlocked

Test Case Task ID: #111
Test Scenario: Verify that a Telegram link code can only be used once
Prerequisites: 1. A valid unused link code has been issued
Test Steps: 1. Tester sends the link code to the bot from one chat 2. Tester sends the same link code again from a second chat
Test Data: 1. One valid link code used twice
Expected Results: The first use links the account. The second use is refused and does not link the second chat.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-TEL-004, TelegramLinkDAOTest.consumeCode_isSingleUse

Test Case Task ID: #112
Test Scenario: Verify that blocking the bot stops further messages without causing repeated retries
Prerequisites: 1. A member has linked their Telegram account 2. A notification is queued for that member
Test Steps: 1. Member blocks the bot in Telegram 2. Tester waits for the queued notification to be sent 3. Tester checks the member's Telegram link status
Test Data: 1. A queued notification for a chat that has blocked the bot
Expected Results: The link is marked inactive, no message is delivered, and the system does not keep retrying.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-TEL-005, TelegramOutboxWorkerTest.blockedUserDeactivatesTheLink

Account: Management and Personal Data

Test Case Task ID: #113
Test Scenario: Verify that a phone number and address are stored encrypted when a profile is updated
Prerequisites: 1. Member is logged in 2. Database can be accessed
Test Steps: 1. Member opens the profile page 2. Member enters a new phone number and address 3. Member clicks Save 4. Tester reads the stored values from the database
Test Data: 1. A new phone number 2. A new address
Expected Results: The profile page shows the new values, and the stored values are not readable as plain text in the database.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ACC-001, TestUpdateProfileServlet.update_reencryptsPiiBeforePersist

Test Case Task ID: #114
Test Scenario: Verify that a member cannot open another user's account page by changing the address
Prerequisites: 1. Member is logged in 2. The identifier of another user's account is known
Test Steps: 1. Member opens their own account page 2. Member adds another user's identifier to the page address 3. Member reloads the page
Test Data: 1. Another user's account identifier
Expected Results: The member's own account details are shown. The identifier in the address is ignored and no other user's data is displayed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ACC-002, TestAccountManagementServlet.loadsOnlySessionUserIdIgnoresRequestParam

Test Case Task ID: #115
Test Scenario: Verify that closing a seller account does not strand a buyer who has already paid
Prerequisites: 1. A seller has a sale that is paid but not yet despatched 2. The seller requests account closure
Test Steps: 1. Seller opens the account settings page 2. Seller requests account closure and confirms 3. Admin opens the refund queue 4. Admin checks their notifications
Test Data: 1. A paid order that has not been despatched
Expected Results: A refund decision is raised for that order and the admins are alerted, so the buyer's payment is not lost.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ACC-003, UserDAOClosureTest.paidUndespatchedSaleRaisesRefund

Test Case Task ID: #116
Test Scenario: Verify that account closure either completes fully or leaves the account untouched
Prerequisites: 1. A member account is eligible for closure 2. A step of the closure process is made to fail 3. Database can be accessed
Test Steps: 1. Member requests account closure 2. The closure process fails part way through 3. Tester inspects the account record
Test Data: 1. An account closure that fails at the cleanup step
Expected Results: The whole closure is rolled back. The account is left as it was, with no partly anonymised details.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-ACC-004, UserDAOClosureTest.cleanupFailureRollsBackAnonymisation

Frontend: Web Interface

Test Case Task ID: #117
Test Scenario: Verify that a guarded page cannot be opened without logging in
Prerequisites: 1. No user is logged in
Test Steps: 1. Guest enters the address of a guarded page such as the watchlist 2. Guest loads the page
Test Data: 1. The address of a guarded page
Expected Results: The guest is redirected to the login page and the guarded content is not shown.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-FE-001, ProtectedRoute.test.jsx, sends a signed-out visitor to the login page

Test Case Task ID: #118
Test Scenario: Verify that a member without selling turned on is offered the selling opt-in rather than an error
Prerequisites: 1. Member is logged in 2. Selling is not turned on for that account
Test Steps: 1. Member opens a seller page such as the seller dashboard
Test Data: 1. A member account with selling not turned on
Expected Results: The selling opt-in page is shown, explaining how to turn selling on. No error page is displayed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-FE-002, ProtectedRoute.test.jsx, offers the selling gate to a member who has not switched selling on

Test Case Task ID: #119
Test Scenario: Verify that the why this panel appears only on recommended listings
Prerequisites: 1. Buyer is logged in 2. The home page shows both recommended and ordinary listings
Test Steps: 1. Buyer opens the home page 2. Buyer checks a card in the recommendations strip 3. Buyer checks a card in an ordinary listing strip
Test Data: 1. One recommended listing card 2. One ordinary listing card
Expected Results: The why this panel is offered on the recommended card only, and does not appear on the ordinary card.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-FE-003, AuctionCard.test.jsx, only shows the why this panel for a recommended card

Test Case Task ID: #120
Test Scenario: Verify that a buyer can see why an item was recommended
Prerequisites: 1. Buyer is logged in 2. The home page shows recommendations
Test Steps: 1. Buyer opens the home page 2. Buyer clicks the why this link on a recommended card
Test Data: 1. A recommended listing card
Expected Results: The panel opens and shows the recommendation score together with the main reason behind it.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-FE-004, AuctionCard.test.jsx, shows the re-ranker score and the component behind it once expanded

Test Case Task ID: #121
Test Scenario: Verify that the countdown changes colour as an auction approaches its end time
Prerequisites: 1. Three auctions exist, ending in more than a day, in less than a day, and in less than an hour
Test Steps: 1. Buyer opens the search results page showing all three auctions 2. Buyer compares the countdown colour on each
Test Data: 1. An auction ending in more than a day 2. An auction ending in less than a day 3. An auction ending in less than an hour
Expected Results: The countdown is shown in the neutral colour, amber and red respectively.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-FE-005, CountdownTimer.test.jsx, colours by urgency: normal, then amber inside a day, then red in the last hour

Test Case Task ID: #122
Test Scenario: Verify that a dialog can be operated by keyboard
Prerequisites: 1. Member is logged in 2. A page with a dialog such as a confirmation prompt is open
Test Steps: 1. Member opens the dialog using the keyboard 2. Member presses Tab to move through the dialog 3. Member closes the dialog
Test Data: 1. A dialog opened from a button on the page
Expected Results: Focus moves into the dialog when it opens, stays within it while tabbing, and returns to the button that opened it when it closes.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-FE-006, Modal.test.jsx, moves focus to the first focusable element on open

Test Case Task ID: #123
Test Scenario: Verify that the card editing screen does not offer the card number for editing
Prerequisites: 1. Buyer is logged in 2. The buyer has a saved card
Test Steps: 1. Buyer opens the payment methods page 2. Buyer opens the saved card for editing
Test Data: 1. A saved card
Expected Results: No card number field is offered, and the page explains that the card number cannot be changed.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-FE-007, AccountSettings.test.jsx, does not offer the card number for editing, and says why

Test Case Task ID: #124
Test Scenario: Verify that live price updates stop when the buyer leaves the auction page
Prerequisites: 1. Buyer is logged in 2. An auction detail page is open with the browser developer tools on the network tab
Test Steps: 1. Buyer watches the repeating price requests on the network tab 2. Buyer navigates away to another page 3. Buyer watches the network tab again
Test Data: 1. An open auction detail page polling for price updates
Expected Results: The repeating requests stop as soon as the page is left, and any request still in progress is cancelled.
Actual Results: As expected
Test Status - Pass/Fail: Pass
Automated test reference: TC-FE-008, usePolling.test.jsx, stops polling and aborts the in-flight request on unmount